LUT core-module ablation: three generations of lookup math

Comparison table, version 1

2026-09-11

Setup and notation

Shared model. LUT transformer, $E = 384$, 6 layers, 6 attention heads. The FFN is `CompressionMultiHeadLUT` with $H = 8$ heads, $tph = 64$ tables per head, $d_{in} = d_{out} = 48$: `compress` $E \to Hd_{in}$, the *core LUT* on $[T, H, d_{in}] \to [T, H, d_{out}]$, `decompress` $Hd_{out} \to E$. Only the core LUT differs between rows. Per head h, the core reads $z_h \in \mathbb{R}^{d_{in}}$.

Per table t (of the tph tables of head h): n fixed anchor pairs $(a_j, b_j) \subset [0, d_{in})$, $u_j = z_h[a_j] - z_h[b_j]$, bits $\beta_j = [u_j > 0]$, cell $c = \text{pack}(\beta) \in [0, K)$ with $K = 2^n$, table $W_t \in \mathbb{R}^{K \times d_{out}}$. $c^{(j)}$ is c with bit j flipped, and $j^\star = \arg\min_j |u_j|$ is the least-confident pair. The head output is $y_h = \sum_{t \in h}(\cdot)$. In the backward, $g = \partial L / \partial y_h$, $G_t(k) = \langle g, W_t[k] \rangle$, and any $\partial L / \partial u_j$ is scattered as $+$ to $z_h[a_j]$ and $-$ to $z_h[b_j]$. σ is the logistic sigmoid. The cell index is integer-valued: no gradient ever flows through it (no straight-through estimator in any row).

Alternatives k**.** Throughout, k counts the table cells whose rows enter the computation, *including* the addressed cell: "two alternatives" is $k = 2$ (c and one flip), "all alternatives" is $k = K$. **The cost also depends on** n (anchor pairs per table), which the variable list in the brief does not include. All concrete numbers use $n = 8$ ($K = 256$), the value in the existing Gen-3 runs, and $P = H \cdot tph = 512$ tables per token.

Counting convention for FLOPs and MACs

- **Scope:** the core LUT module only, **per token**, one layer. Multiply by T for a sequence and by 6 for the model. The `compress`/`decompress` projections are identical in every row and excluded: they add $E \cdot Hd_{in} + Hd_{out} \cdot E = 294{,}912$ MACs ($\approx 589{,}824$ FLOPs) forward and about twice that backward per token per layer, which is *more* than the forward of every core variant below.
- **FLOP** = one scalar floating-point operation: $+, -, \times, \div$, a float comparison, $|\cdot|$, exp, $\log \sigma$, σ each count 1. **MAC** = a multiply whose result is accumulated (weight $\times$ value, then added): **1 MAC = 2 FLOPs**, and MACs are included in the FLOP column. A pure accumulate (adding a looked-up row) is 1 FLOP and not a MAC.
- **Not counted:** memory reads (table-row gathers, anchor gathers), integer work (bit packing, XOR, index offsets), optimizer updates, and saving activations. Row reads per token are $P \cdot k$ rows of d_{out} values where they matter.
- **Forward** = inference cost (what an evaluated model executes). **Backward** = all *additional* arithmetic a training step needs, including training-only forward work (for example the alternative search in 1.1).
- Terms proportional to d_{out}, K and nK are exact. Scalar per-table overheads ($O(n)$ and constants) are counted from the code but approximate to within a few operations per table, since autograd's internal breakdown varies. d_{in} does not enter the core cost: it only sets the anchor index range.
- Temperature gradients ($O(1)$ per layer) are ignored. Gen-2 rows assume the confidence gate is *off*, as in the existing Gen-2 runs.

Table A — forward and backward math

Variant	Forward pass (math)	Backward pass (math)
GEN 1 — Spiking Manifesto basic model, rational uncertainty $U(u) = 0.5/(1+\lvert u\rvert)$[1]		
1.1 Hard forward, two-alternatives backward ($k=2$)	$y_h = \sum_t W_t[c_t]$. Training only: $j^\star = \arg\min_j \lvert u_j\rvert$ (for $k>2$: the $k-1$ smallest $\lvert u_j\rvert$).	Weights: $\partial L/\partial W_t[c_t]$ += g (addressed row only). Input (surrogate — the derivative of 1.2's blend): $\partial L/\partial u_{j^\star} = -U'(u_{j^\star})\left(G_t(c_t) - G_t(c_t^{(j^\star)})\right)$, with $-U'(u) = \dfrac{0.5\ \mathrm{sgn}\,u}{(1+\lvert u\rvert)^2}$; $\partial L/\partial u_j = 0$ for $j \neq j^\star$. For $k>2$ each flip contributes and the sum is divided by $k-1$.
1.2 Soft forward, two-alternatives backward ($k=2$)	$y_h = \sum_t \left[(1-U_t)\,W_t[c_t] + U_t\,W_t[c_t^{(j^\star)}]\right]$, $U_t = U(u_{t,j^\star})$. Continuous at $u_{j^\star}=0$ (both weights $\frac{1}{2}$). Applied at eval too.	Weights: $\partial L/\partial W_t[c_t]$ += $(1-U_t)\,g$, $\partial L/\partial W_t[c_t^{(j^\star)}]$ += $U_t\,g$. Input: same expression as 1.1, which here is the *exact* gradient of the blend with the two cells held fixed.
GEN 2 — FastMHL math. Surrogate: $\rho_j = \dfrac{\lvert u_j\rvert}{T_s + \lvert u_j\rvert}$, $s_k = \sum_j \rho_j - 2\sum_{j:\,\mathrm{bit}_j(k)\neq\beta_j}\rho_j$, $\pi_k = \mathrm{softmax}_k(s_k/T_{sel})$ (signs β pinned; T_s, T_{sel} are temperatures)		
2.1 Hard forward, all-alternatives backward ($k=K$)	$y_h = \sum_t W_t[c_t]$. ($c_t = \arg\max_k s_k$; no K-sized tensor is built.)	Weights: $\partial L/\partial W_t[c_t]$ += g. Input via the K-cell surrogate $\hat{y}_t = \sum_k \pi_k W_t[k]$ (never used in the forward): $\dfrac{\partial L}{\partial u_j} = \dfrac{1}{T_{sel}}\sum_k \pi_k\left(G_t(k) - \bar{G}_t\right)\dfrac{\partial s_k}{\partial \rho_j}\cdot\dfrac{T_s\ \mathrm{sgn}\,u_j}{(T_s+\lvert u_j\rvert)^2}$, $\bar{G}_t = \sum_k \pi_k G_t(k)$, $\partial s_k/\partial\rho_j = \pm 1$ (agree/disagree with β_j). All n pairs get gradient; reads all K rows.
2.2 Hybrid_smooth two-alternatives forward, all-alternatives backward (fwd $k=2$, bwd $k=K$)	$y_h = \sum_t \left[(1-w_t)W_t[c_t] + w_t W_t[c_t^{(j^\star)}]\right]$, $w_t = \sigma(-\Delta_t/T_{sel})$, $\Delta_t = 2\rho_{t,j^\star}$ (the exact 2-way softmax of s over $\{c, c^{(j^\star)}\}$). Applied at eval too.	Weights: $(1-w_t)\,g$ into $W_t[c_t]$ and $w_t\,g$ into $W_t[c_t^{(j^\star)}]$. Input: the K-cell surrogate of 2.1 (not the gradient of the forward that actually ran).
2.3 Hard forward, 2-alternatives backward	$y_h = \sum_t W_t[c_t]$	**TBD** — not implemented. `FastMultiHeadLut` raises `ValueError` for `backward_topk>0` unless `forward_mode='hybrid_smooth'` (`fast_multi_head_lut.py:1305`).
2.4 Hybrid_smooth two-alternatives forward, 2-alternatives backward ($k=2$)	As 2.2.	Weights: as 2.2. Input (`backward_topk=1`): the exact gradient of the 2-cell blend with cells fixed. Only $j^\star$ gets gradient: $\dfrac{\partial L}{\partial u_{j^\star}} = -\dfrac{w_t(1-w_t)}{T_{sel}}\left(G_t(c_t^{(j^\star)}) - G_t(c_t)\right)\dfrac{2T_s\ \mathrm{sgn}\,u_{j^\star}}{(T_s+\lvert u_{j^\star}\rvert)^2}$.
GEN 3 — LightMHL math (LookupFFN-inspired). Margin score $s_t = \left(\sum_j \lvert u_{t,j}\rvert\right)\prod_j \sigma(2\lvert u_{t,j}\rvert)$; the cell index uses detached u[2]		
3.1 Soft forward n=1, margin confidence, single-alternative backward ($k=1$)	$y_h = \sum_t s_t\,W_t[c_t]$. One hard cell read scaled by a differentiable score: y jumps at every cell boundary.	Weights: $\partial L/\partial W_t[c_t]$ += $s_t\,g$. Input (only through the score): $\dfrac{\partial L}{\partial u_j} = G_t(c_t)\left(\prod_i \sigma(2\lvert u_i\rvert) + 2s_t\,\sigma(-2\lvert u_j\rvert)\right)\mathrm{sgn}\,u_j$ for all j. No other cell is compared.
3.2 Soft forward n=2, margin confidence, two-alternatives backward ($k=2$)	$y_h = \sum_t s_t\left[\omega_0 W_t[c_t] + \omega_1 W_t[c_t^{(j^\star)}]\right]$, $\omega_1 = \sigma(-2\lvert u_{j^\star}\rvert/\tau)$, $\omega_0 = 1-\omega_1$, $\tau = e^{\log\tau}$ (learnable).	Weights: $s_t\omega_0\,g$ and $s_t\omega_1\,g$ into the two read cells. Input: $\dfrac{\partial L}{\partial u_j} = (\omega_0 G_0 + \omega_1 G_1)\dfrac{\partial s_t}{\partial\lvert u_j\rvert}\mathrm{sgn}\,u_j + [j=j^\star]\,s_t\,\frac{2}{\tau}\omega_0\omega_1(G_0-G_1)\,\mathrm{sgn}\,u_j$, with $G_0 = G_t(c_t)$, $G_1 = G_t(c_t^{(j^\star)})$. $\partial L/\partial\log\tau = \sum_t s_t\omega_0\omega_1(G_1-G_0)\,2\lvert u_{j^\star}\rvert/\tau$.

[1] Other uncertainty functions may be tried later. The code also offers `UncertaintyMode.INVERSE_QUADRATIC`, $0.5/(1+u^2)$.
[2] A hard-forward Gen-3 variant still needs to be designed: nothing in the code trains LightMHL with a hard forward.

Table B — cost and results

Per token, one layer, core LUT only. Concrete values at $H = 8$, $tph = 64$, $d_{in} = d_{out} = 48$, $n = 8$ ($K = 256$), $P = H\,tph = 512$, $d = d_{out}$. Validation bpb is on 16K steps, batch 48 rows × 512 tokens.

Variant	Forward FLOPs	Backward FLOPs	Forward MACs	Backward MACs	Validation bpb (16K, bs48)	Implementation (class used)
1.1	$P(2n+d)$ = **32,768**	$P\big(2kd+d+2n+9(k-1)\big)$ = **135,680**	0	$P\,kd$ = **49,152**	**TBD** — not yet implemented in the FFN harness; no run exists.	`spiky.lutorch.multi_head_lut.MultiHeadLut` (`smooth_mode=False`, `n_alternatives=1`, `uncertainty_mode=INVERSE_L1`); also `lut_fused.LUTLayer.LUTLayerBasic`. **Not wired into** `CompressionMultiHeadLUT`.
1.2	$P(2kd+4n+3k-2)$ = **116,736**	$P\big(4kd+9(k-1)\big)$ = **201,216**	$P\,kd$ = **49,152**	$P\,2kd$ = **98,304**	**TBD** — not yet implemented in the FFN harness; no run exists.	`MultiHeadLut` (`smooth_mode=True`, `n_alternatives=1`). Not wired into `CompressionMultiHeadLUT`.
2.1	$P(2n+d)$ = **32,768**	$P(2Kd+4nK+9K+11n+d)$ = **18,026,496**[3]	0	$P(Kd+2nK)$ = **8,388,608**	**TBD** — not yet run on the paper protocol. Prior: **1.2281** (`exp_g_0014`), n=6, old eval[4]	`CompressionMultiHeadLUT(lut_impl='fast')` → `FastMultiHeadLut(forward_mode='hard', backward_topk=0)`; keys `lut_forward_mode`, `lut_backward_topk`.
2.2	$P(2kd+4n+5)$ = **117,248**	$P(2Kd+4nK+9K+11n+2kd)$ = **18,100,224**[3]	$P\,kd$ = **49,152**	$P(Kd+2nK+kd)$ = **8,437,760**	**TBD** — not yet run on the paper protocol. Prior: **1.2176** (`exp_n_0044`), n=6, old eval[4]	`FastMultiHeadLut(forward_mode='hybrid_smooth', backward_topk=0)`.
2.3	$P(2n+d) = 32{,}768$	n/a	0	n/a	**TBD** — not yet implemented	Would be `FastMultiHeadLut(forward_mode='hard', backward_topk=1)`, which the code rejects.
2.4	$P(2kd+4n+5)$ = **117,248**	$P(4kd+9n+15)$ = **241,152**	$P\,kd$ = **49,152**	$P\,2kd$ = **98,304**	**TBD** — not yet run (the only `backward_topk>0` run, `exp_n_0188`, is H=4, tph=128, n=7).	`FastMultiHeadLut(forward_mode='hybrid_smooth', backward_topk=1)`. Note: `backward_topk` counts cells *beyond* the addressed one.
3.1	$P(2kd+7n)$ = **77,824**	$P(4kd+8n)$ = **131,072**	$P\,kd$ = **24,576**	$P\,2kd$ = **49,152**	**TBD** — not yet run at 16K. 48K: **1.1313** (`exp_g_0207`)[5]	`CompressionMultiHeadLUT(lut_impl='light')` → `LightMultiHeadLUT(confidence_form='margin', read_top_n=1)`.
3.2	$P(2kd+8n+7)$ = **134,656**	$P(4kd+8n+11)$ = **235,008**	$P\,kd$ = **49,152**	$P\,2kd$ = **98,304**	**TBD** — not yet run at 16K. 48K: **1.1304** (`exp_g_0206`)[5]	`LightMultiHeadLUT(confidence_form='margin', read_top_n=2, read_tau=0.5, learnable)`.

[3] Dominated by $2Kd$: the all-alternatives backward reads all K rows of every table. At the existing runs' $n = 6$ ($K = 64$) the 2.1 backward is $P \cdot 8{,}370 = 4{,}285{,}440$ FLOPs. Memory: at $n = 8$ the backward holds several `[tokens, 512, 256]` fp32 buffers, about 12.9 GiB each at 24,576 tokens, so batch 48 needs micro-batching on a 32 GB card.

[4] Old batch-coupled eval (245,760 val tokens from token 0, no row skip), $n = 6$, tied embeddings, learnable temperatures; branch `research/hyperplane_ffn_next`. **Not comparable** to the corrected protocol. Same-protocol neighbours: `exp_n_0052` 1.2286 (hard, batched temperatures).

[5] 48,000-step run, $n = 8$, untied, fixed eval; device batch 12 with gradient accumulation to 24,576 tokens. **Not a 16K result.** The step-16,000 rows (1.1862 for n=1, 1.1793 for n=2) sit on a 48K cosine schedule and are not comparable either.

Where the code disagrees with the brief

1. **Gen 1 is not wired into the experiment harness.** `CompressionMultiHeadLUT` accepts only `lut_impl` ∈ {`fast`, `light`, `bh4`} (`compression_mhl.py:184`), and no `model_build.py` key reaches `MultiHeadLut`. `MultiHeadLut` also takes a shared `[B, input_dim]` input, not the per-head `[T,H,d_in]` layout, so rows 1.1/1.2 need new wiring (for example `anchor_candidates` restricting each head's anchors) before they can run.
2. **In 1.1 the input gradient is a surrogate.** It is the derivative of 1.2's soft blend, while the weights update only the addressed row.
3. **The paper's status report writes** $u(\delta) = 0.5/(T + |\delta|)$ with a per-LUT temperature. The code's $U(u) = 0.5/(1 + |u|)$ has no temperature (`l_projection.py:96`, `lutorch.cu:797`).
4. **2.3 cannot be built**: hard forward with a sparse backward raises `ValueError` (`fast_multi_head_lut.py:1305`).
5. **`backward_topk` is off by one relative to "k alternatives".** It counts cells beyond the addressed one: "2 alternatives" is `backward_topk=1`, and `backward_topk=2` means 3 cells.
6. **"All alternatives" in Gen 2 is all** $K = 2^n$ **cells** (a full softmax surrogate), not the n single-bit flips (that would be `backward_topk=n`). `backward_topk` never changes the weight gradient, which always uses the rows the forward read.
7. **Gen 2's hybrid_smooth weights are** $w = \sigma(-2\rho_{j^*}/T_{sel})$, a temperature-controlled rational soft-sign, not Gen 1's $U(u)$. It does match "two-alternatives forward".
8. **3.1 "soft forward** n=1**" is not soft in the cell sense.** It reads one hard cell and scales it by a differentiable margin score, and its backward compares no alternative cell. "Single-alternative backward" is accurate only in the sense that the input gradient flows through the score alone.
9. **The margin confidence function is LookupFFN's kernel** $s = (\sum_j |u_j|) \prod_j \sigma(2|u_j|)$, not $\min_j |u_j|$. The factor 2 is fixed.
10. **The cost depends on** n **(anchor pairs), which the brief's variable list omits.** Existing Gen-2 16K runs use $n = 6$, Gen-3 runs use $n = 8$, so a paper set needs one n for all rows.
11. **Minor:** `exp_g_0207` (3.1) sets `lut_read_tau_learnable=true`, which creates a parameter that never receives a gradient at $n = 1$.

TBD cells and experiments still to run

- **Needs implementation first:** 1.1 and 1.2 (wire `MultiHeadLut` into `CompressionMultiHeadLUT`); 2.3 (a hard-forward path for `backward_topk`).
- **Needs a run on the paper protocol** (16K steps, batch 48, corrected eval, one fixed n): 2.1, 2.2, 2.4, 3.1, 3.2. The 2.1/2.2 numbers above are $n = 6$ on the old eval, and 3.1/3.2 exist only at 48K.
- **Every row's validation bpb is TBD on the paper protocol.** Cost cells for 2.3 are n/a until it exists.
- **Reference anchors** (vanilla dense FFN, same protocol, corrected eval): 1.1651 / 1.1618 at 16K (two seeds), 1.1154 at 48K.